

Tutor Handbook

Optimal Market Making for Cryptocurrency Options — The
Avellaneda–Stoikov Framework and its Extension to Options

North Carolina State University · Financial Mathematics

Summer 2026

NC STATE UNIVERSITY

Tutor Handbook

Optimal Market Making for Cryptocurrency Options

The Avellaneda–Stoikov Framework and its Extension to Options

A 10-Week Summer Research Program — North Carolina State University

Prepared for teaching assistants, recitation leaders, and graders.

Program Mentor: Dendi Suhubdy (Bitwyre). Guest Practitioner: Fenni Kang (Coincall). Edition: 2026.

How to Use This Handbook

This handbook is the operational companion to the student-facing syllabus. Read the syllabus first. The handbook has four parts: Part I (orientation — your role, the weekly rhythm, policies), Part II (the week-by-week teaching guide), Part III (milestone evaluation rubrics), and Part IV (administrative reference). The detailed per-problem solution sketches and grading rubrics now live in the separate **Solutions** booklet; this handbook tells you how to *run* each week and how to grade consistently, and points you to the Solutions booklet for the worked answers.

Conventions. Boldface marks the single most important point in a section. A [!] marks situations to escalate to the mentor. Time allocations are recommendations; adapt to the cohort.

A note on the two languages. Problem sets are written in **Python** with **PyTorch**; differentiation is done by autograd, and students are *not* asked to hand-derive AD. The fast-computation week (Week 7) is written in **C++** and exposed to Python via a **pybind11** binding. When you grade or help, hold students to “PyTorch for research and Greeks, C++ for the hot path, **pybind11** between them.” If a student hand-rolls adjoint AD in C++, that is *not* required and usually a sign they have over-engineered — steer them back to autograd as the reference.

Part I. Tutor Orientation

1.1 Your Role

Tutors deliver the day-to-day program: leading the weekly ninety-minute recitation, holding three weekly office hours, grading weekly problem sets within seven days, giving first-pass evaluation of milestone deliverables (final evaluation rests with the mentor), and flagging at-risk students to the mentor within twenty-four hours. Tutors do **not** deliver the main lectures or unilaterally change the syllabus, problem sets, or rubrics — raise proposed changes with the mentor.

1.2 Relationship with the Mentor

The program is mentored by Dendi Suhubdy, who sets direction, delivers the main lectures, and makes final decisions on grading disputes and milestone evaluations. Standing channels: a weekly thirty-minute tutor–mentor sync; a shared Slack workspace (mentor responds within 24 business hours); and a direct phone line for crises ([!] student in acute distress, suspected integrity violation, data-security incident) — use sparingly.

Note that two lectures are delivered by a **guest practitioner**, Fenni Kang (Chief Strategy Officer, Coincall): the taker-strategies session in Week 9 and the structured-products session in Week 10. Coordinate logistics for these with the mentor; you support them in recitation as usual.

1.3 Weekly Rhythm

For a cohort of six to eight students, a typical week is: Sunday evening prep (read this handbook’s Part II section, ~2–3 h); attend Lecture 1 (Mon) and

Lecture 2 (Wed) to track confusions; office hours (Tue, Fri, Sat mornings); lead recitation (Thu); grade the previous week's problem set over the weekend (~4–6 h). Total $\approx 15\text{--}20$ h/week, peaking in milestone weeks (5, 7, 9), when you also join code review.

1.4 Office Hours Guidance

- **Resist solving problems for students.** Ask what they tried, ask what the next step is, give the smallest unblocking hint.
- Distinguish conceptual confusion (return to lecture) from implementation confusion (read the code together).
- The same student with the same confusion every week is a signal — flag it to the mentor.
- Office hours are not milestone code review. Do not debug milestone code in office hours; that happens in scheduled code-review sessions during milestone weeks, so your grading judgment stays independent.

1.5 Grading Standards

Consistency matters more than the calibration of any single grade. With two tutors, grade the first problem set's first three problems together to calibrate, and spot-check each other thereafter.

Every problem set is a programming assignment. Specific standards:

- **Code is graded on correctness, reproducibility, and code quality, in that order.** Code that runs and produces the right answer with poor style is graded above code that does not run.
- **Numerical-verification problems** (confirm an analytical result by computation) are graded on whether the verification is set up correctly and the reported error is the right order of magnitude — not on a written proof.
- **Short written interpretations** (typically printed by the code as a summary of computed results) are graded on substantive engagement with the actual numbers; restating the question earns zero.
- For Week 7, additionally grade the **C++/pybind11 boundary**: does the module build with CMake, import cleanly, and agree with the PyTorch reference Greeks?

Return graded work within seven days. Provide written feedback on at least one problem per set, even at full marks. The worked solutions are in the Solutions booklet; develop your own complete solution before grading.

1.6 Tutor Calibration Session

Before the program, the mentor holds a half-day calibration session: review the syllabus, walk through one full week of these notes, grade a sample problem set as a group, and review the milestone rubrics. Attendance is mandatory.

Part II. Week-by-Week Teaching Notes

Each week section gives the spine of the week, lecture-emphasis notes, common student confusions, and a recitation plan. Read the upcoming week in full during the Sunday prep block. Grading rubrics and worked answers are in the Solutions booklet.

Week 1 — Introduction (Delivered)

This week was delivered last week. It oriented the cohort, set up the Python + PyTorch and C++17/CMake/pybind11 toolchains, distributed Coincall data credentials, and collected the data-use agreements. There is no graded problem set. If you are onboarding mid-stream, confirm every student can (a) run a PyTorch autograd “hello world” and (b) build and import the pybind11 starter module — these are prerequisites for Weeks 5 and 7 respectively.

Week 2 — Foundations of Market Microstructure

THE SPINE. Students must leave able to think of a market maker as a continuously quoting participant justified by the spread, and able to read a limit order book without looking up vocabulary. Everything builds on this.

Lecture emphasis. Lecture 1 is vocabulary and institutional fluency; the key under-stated point is that price-time priority creates a queue and queue position is a real economic asset (this matters in Week 5 fill calibration). Lecture 2 covers Roll, Glosten–Milgrom, and Kyle quickly — the Glosten–Milgrom decision tree drawn on the board is the most important artifact; have students copy it. Kyle is for completeness.

Common confusions. Why quote both sides rather than react? (A maker commits in advance; the spread compensates for that.) Aren’t all orders limit orders? (Mathematically yes, institutionally no — different fees and priority.) The Roll estimator differs from the observed spread — is that a bug? (No — it estimates the unobserved component; the gap is the finding.)

Recitation. Goal: every student leaves with a working Python class that reconstructs an L2 book from an event stream. Materials: a one-hour Coincall

BTC-PERP event file (JSON), a tutor-only reference solution, a projector. Timing: greet/recap (0–10); live demo of loading the data and one minute of manual reconstruction (10–20); pair programming on the scaffold (20–60) — the most common bug is failing to clean up empty price levels after a cancel; reconvene and debug one pair’s code together (60–75); wrap up lecture-flagged questions (75–90). Save the working code; it is reused all program.

Week 3 — The Avellaneda–Stoikov Model

THE SPINE. Students must leave with the full stochastic-control pipeline as a worked example: dynamics $\rightarrow$ HJB $\rightarrow$ ansatz $\rightarrow$ ODE reduction $\rightarrow$ closed-form solution. Weeks 4–5 are variations on this template.

Lecture emphasis. Lecture 1’s two key modelling choices: arithmetic (not geometric) Brownian motion for the mid, and exponential (CARA) utility, both for tractability — make sure students see these as choices. Lecture 2’s centerpiece is the separation ansatz $V = -\exp(-\gamma(x+qS)) \cdot \theta(t,q)$; it recurs in every later control problem, so attendance matters most here.

Common confusions. Why CARA? (Tractability — it decouples the value function from wealth.) The ansatz seems to come from nowhere (it is principled — multiplicative structure under CARA + arithmetic dynamics; accept on faith, it recurs). Why is the half-spread roughly symmetric but the reservation price asymmetric? (Different quantities — the reservation price is the indifference point; the half-spread is the band width.) Huge simulated P&L? (The model assumes you are alone with exogenous flow; Weeks 5 and 9 add realism.)

Recitation. Group whiteboard derivation (0–30) — the single most important hour for many students; take your time. Then implement the closed-form quotes in PyTorch (30–60) and run a γ sensitivity sweep (60–80), aggregating inventory variance and Sharpe on the projector; discuss where you would set γ for a real desk (80–90).

Week 4 — Stochastic Optimal Control in Continuous Time

THE SPINE. Students leave with the abstract template — dynamic programming principle, HJB, verification theorem — and the ability to recognize when a problem fits it. The Merton problem reinforces, not introduces, the Week 3 pattern.

Lecture emphasis. Lecture 1 derives HJB in generality with Avellaneda–Stoikov as a special case; the infinitesimal generator often needs re-explaining for weaker stochastic-calculus backgrounds. Lecture 2 states the verification theorem (the missing Week-3 piece) and contrasts CARA with CRRA. Do not let students get stuck on viscosity solutions — not central here.

Common confusions. Why is HJB necessary? (It encodes the DPP, which is necessary by definition; the verification theorem gives sufficiency.) Why does wealth appear under CRRA but not CARA? (Constant marginal utility of wealth under CARA.) Verification vs. existence theorem? (We always have a candidate, so we only need verification.)

Recitation. Derive Merton under CARA (10–45), then CRRA (45–75), faster the second time; side-by-side comparison (75–90). If the cohort is strong, compress CARA and spend time on Almgren–Chriss instead.

Week 5 — The Avellaneda–Stoikov Engine → Milestone 1

THE SPINE. Two things: students understand how the closed form becomes running code, and they submit Milestone 1, the linear-asset Avellaneda–Stoikov engine in Python/PyTorch.

Lecture emphasis. Lecture 1 is architecture and the move from continuous time to discrete events; emphasize calibrating $\lambda(\delta)=A\cdot\exp(-\kappa\delta)$ by maximum likelihood rather than a naive curve fit. Lecture 2 is calibration and validation, and formally announces the milestone and rubric.

Common confusions. Calibrated intensities look implausible (crypto fill data is noisy; fit on one week, then two, then four — estimates should stabilize). Backtest P&L is negative (check: spread too tight vs. calibrated intensities, inventory penalty too small, or a regime shift in the window — before suspecting the model). My PyTorch engine is slow (vectorize over inventory; avoid Python loops over the grid).

Recitation. [!] Milestone week — block four extra hours of code review. Whiteboard any remaining derivation gaps (0–25), then live code review of one volunteer’s engine (25–70) — this reveals cohort-wide bugs you address in the wrap-up — then open Q&A on the milestone acceptance criteria (70–90).

Week 6 — Options Pricing, the Volatility Surface, and Calibration

THE SPINE. Two tracks: conceptually, implied-vol surfaces and their no-arbitrage constraints; practically, calibrating SVI and SABR to messy Coincall data. Students should leave knowing calibration is half craft, half theory.

Lecture emphasis. Lecture 1 spends the most time on calendar and butterfly arbitrage (the constraints naive parameterizations violate); vanna/volga are introduced but not tested until Week 8. Lecture 2 live-demos a calibration that first fails — the diagnostic process is more valuable than the eventual fit. Students fit using PyTorch autograd for the Jacobians, so they need not derive gradients by hand.

Common confusions. Why two parameterizations? (They encode different priors and fail in different regimes.) Huge residuals on far-OTM puts? (Wide

spreads / low volume — weight by volume or inverse spread.) SVI params jump day to day? (Often the initial guess — seed from yesterday’s fit.) SABR vs. Monte Carlo mismatch at long maturities? (Hagan is a small-time expansion; MC is the truth.)

Recitation. Every student produces a working SVI and SABR calibration with diagnostics on a chosen snapshot. Keep a backup, slightly messier snapshot ready in case the chosen one is too clean for the “crypto calibration is hard” lesson to land.

Week 7 — Fast Computation (Prices, Greeks, Implied Vol, Surfaces) → Milestone 2

THE SPINE. The week is about latency. Students must internalize that a correct but slow pricer is not enough. The tools are characteristic- function pricing, robust implied-vol inversion, and — the new emphasis — **binding C++ to Python with pybind11**. The end is a sub-5ms surface revaluation callable from Python.

Lecture emphasis. Lecture 1: COS is the workhorse (Carr–Madan for context); the truncation interval must be computed from the cumulants, not guessed; robust implied-vol inversion needs a good initial guess (Jäckel-style) plus a safeguarded Newton/Halley step. Lecture 2 is the **binder lecture**: structure a `pybind11` module, build it with CMake, pass NumPy/PyTorch tensors without copying (buffer protocol / `py::array_t`), release the GIL around the hot loop, and validate C++ Greeks against PyTorch autograd. **Emphasize: autograd is the oracle for Greeks; C++ is for speed.** We deliberately avoid hand-written adjoint AD in C++ — students found it confusing and it is unnecessary when autograd provides the reference.

Common confusions. “Why are CF methods faster than Black–Scholes?” (They’re not for BS — the point is Heston, where there’s no closed form.) “My `pybind11` build can’t find Python/pybind11 headers.” (CMake `find_package(pybind11)`; use the provided `CMakeLists.txt` scaffold.) “My C++ Greeks disagree with autograd.” (Usually a units or finite-difference-bump issue in the comparison harness — or the GIL not released causing a benchmarking artifact.) “COS won’t match within $1e-4$.” (Truncation interval from cumulants; raise N to 2^{10+} .)

Recitation. [!] Milestone week — block four extra hours of code review. Profile a naive Python pricer (10–40), move the kernel to C++ and bind it with `pybind11` (40–70), re-benchmark and confirm the surface revalues under 5 ms with Greeks matching autograd (70–90). Have the reference timings ready so students know what “good” looks like. The latency target is a hard gate — be ready for arguments and refer to the rubric.

Week 8 — Options Market Making Theory and Dynamic Hedging

THE SPINE. The conceptual centerpiece: options market making is Avellaneda–Stoikov with a vector-valued inventory, and the quoting strategy interacts with an explicit hedging policy. The most cognitively demanding week.

Lecture emphasis. Lecture 1 replaces scalar inventory with a vector of Greeks and the scalar penalty with a quadratic form, derives the multi-Greek HJB, and verifies reduction to Avellaneda–Stoikov in 1-D. Discourage setting the penalty matrix to the identity (defer the “how to choose it” discussion to office hours). Lecture 2 (Taleb) derives the gamma–theta–variance identity — the conceptual key to options trading — and presents external vs. integrated hedging and perpetual futures as the hedge instrument.

Common confusions. Why is inventory a vector? (Aggregated Greeks, not contract positions — long one call and short another at a different strike is zero contracts but nonzero gamma/vega.) How to choose the penalty matrix? (Calibrate to the empirical Greek covariance or a desired risk-equivalence; it is part of the model design.) Hedge with perps or the underlying? (Perps — lower cost and the options are quoted against the perp index; account for funding.)

Recitation. Multi-Greek HJB derivation (0–35), then a PyTorch hedged-book simulation under two policies (35–70) with P&L distributions aggregated on the projector; discuss which policy wins in which regime and tie to the Milestone 3 design (70–90).

Week 9 — Backtesting, Risk Analysis, and Taker Strategies → Milestone 3

THE SPINE. Students assemble everything into a working backtest on three months of Coincall data — more engineering discipline than new theory — and gain a practitioner’s view of taker flow from the guest lecture.

Lecture emphasis. Lecture 1 covers event-driven architecture and the pitfalls (look-ahead, unrealistic fills, expiry handling) and the attribution sum-to-total check (a mismatch is always a bug). **Lecture 2 is the guest session (Fenni Kang) on taker strategies** — when/why takers cross the spread, the signals behind aggressive flow, and what taker behaviour implies for order-flow toxicity. Brief students beforehand to come with one question linking taker flow to their own fill simulator.

Common confusions. Suspiciously high P&L? (Always a bug — look-ahead in the surface, 100% fills, missing costs/funding.) Fill-simulator calibration seems circular? (Use a holdout: calibrate on one month, validate on a disjoint month.) Attribution doesn’t sum to total? (Timing — streams computed from different state snapshots.) Sharpe of 6 over three months? (Estimation error — require a block-bootstrap CI and the deflated Sharpe.)

Recitation. [!] Milestone week — six to eight hours of code review. Each student presents one regime (one week of the window) and walks through the P&L attribution; probe for the pitfalls. This is the dress rehearsal for the milestone.

Week 10 — Advanced Avellaneda–Stoikov, Structured Products, and Final Presentations

THE SPINE. Students choose one advanced Avellaneda–Stoikov extension, implement it in a week, and present the final report. Your role shifts from instruction to mentorship: help each student scope realistically — a clean partial result beats an incomplete grand one.

Lecture emphasis. Lecture 1 surveys the extension menu (multi-asset, adverse selection, signal-driven, discrete inventory, robust); students choose by end of day. **Lecture 2 is the guest session (Fenni Kang) on corporate usage of options and structured products** — collars, accumulators/decumulators, and coupon products — and how that flow lands on the market-maker’s book. It closes with research-communication guidance.

Common confusions. “My extension isn’t working — should I switch?” (No time to restart — write up the smallest version that works and acknowledge limitations.) Report length? (≤ 30 pages, quality over length.) LLM use? (Writing support only, not content generation — see Part IV.)

Recitation. Final dry-run presentations (15 minutes each), cohort feedback, tutor focused on the questions an industry audience would ask. Be tough but constructive — expose problems while there is still time to fix them. Submit feedback rubrics to the mentor.

Part III. Milestone Evaluation Rubrics

Each milestone is graded out of 100. The final milestone grade is the average of two independent evaluations (yours and the mentor’s); discrepancies over 10 points trigger a third reviewer.

3.1 Milestone 1 — Linear-Asset Avellaneda–Stoikov Engine (Week 5)

Criterion	Pts	Description
Correctness	30	PyTorch quotes match the closed-form asymptotic formula within 1e-6 on a synthetic problem with known solution.
Intensity calibration	20	ML calibration with χ^2 goodness-of-fit; student demonstrates awareness of the fit's limitations.
Reproducibility	15	Identical config $\rightarrow$ identical results under a fixed seed. Non-reproducibility is an automatic -15 .
Backtest match	15	P&L within $\pm 50\%$ of the mentor's reference on the same window.
Vectorization	10	Quote computation vectorized over inventory in PyTorch (no Python grid loop).
Code quality	5	Lints; type hints; <code>pytest</code> over the core quote computation.
Report	5	≤ 10 pp.; follows the Avellaneda-Stoikov paper structure; honest limitations.

3.2 Milestone 2 — Fast Pricing and Greeks Library (Week 7)

Criterion	Pts	Description
Pricing accuracy	20	Max relative error 1e-4 vs. Coincall reference across the BTC surface.
Greek accuracy	15	C++ Greeks within 1e-4 of the PyTorch-autograd reference for all six Greeks (delta, gamma, vega, theta, vanna, volga).
Latency (full surface)	25	300–500 contracts revalued with full Greek vector in under 5 ms, one CPU core. Hard gate: failing caps the grade at 70.
pybind11 integration	10	Module builds with CMake, imports cleanly from Python, passes tensors without unnecessary copies.

Criterion	Pts	Description
Implied-vol robustness	10	Inversion converges across the surface including deep wings and short expiries.
Tests	10	Cover all pricers, the implied-vol solver, and the surface adapter, plus one edge case.
Technical note	10	Latency reported as a distribution (histogram), with optimization choices explained.

3.3 Milestone 3 — Full Options Market-Making Backtest (Week 9)

Criterion	Pts	Description
Reproducibility	10	Identical replay → identical results. Non-reproducibility is an automatic −10.
P&L attribution	20	Streams (spread, inventory, gamma, vega, hedging) sum to total P&L within 1 bp.
Risk metrics	15	Sharpe, max drawdown, Calmar with block-bootstrap CIs. The CI requirement is non-negotiable.
Greek-exposure tracking	10	Aggregated Greek time series plotted; exposures within configured limits.
Engine quality	15	Correctly integrates the Week-7 pricing library (via <code>pybind11</code>) with the Week-5 engine.
Hedging integration	10	Dynamic hedge correctly integrated; funding rate accounted for.
Forensic analysis	10	Drawdown forensic identifies specific positions, the environment, and a counterfactual.
Report	10	≤ 20 pp.; includes the forensic; honest failure-mode discussion.

Common bugs to check: look-ahead in the volatility surface (−15 engine quality), 100% fill assumption (−10), funding ignored (−5 hedging), Sharpe without

CI (−8 risk).

3.4 Final Report (25 pts) and Presentation (15 pts)

Report: problem motivation (3), methodology (6), results with uncertainty quantification (6), discussion (4), an explicit limitations section (3), writing quality (3). **Presentation:** structure (3), technical depth and Q&A handling (4 + 2), visual aids (3), delivery (3).

Part IV. Administrative Reference

4.1 Academic Integrity

Follows the NC State Code of Student Conduct. Collaboration on problem sets is encouraged but submissions must be each student's own work (discussing the approach is fine; copying code is not). Cite any external source, including LLM-generated content. LLM assistance is allowed for writing support (grammar, phrasing) but not for technical content generation; disclose non-trivial use. [!] The mentor handles all integrity cases — document specifics and escalate within 24 hours; do not confront the student.

4.2 Late Work

Problem sets are due Sunday 11:59 pm Eastern; late until Tuesday 11:59 pm with a 20% deduction; nothing accepted after. Milestones are due Friday 11:59 pm Eastern of the milestone week; no late milestones. Emergencies (documented medical/family) are handled by the mentor only.

4.3 Accommodations

Implement documented accommodations as filed with the NC State Disability Resource Office; do not ask students to justify them. If unsure how to implement one, ask the mentor.

4.4 Crisis Escalation [!]

Escalate immediately (use the phone line, not Slack): suspected integrity violation; student in acute distress; data-security incident (e.g., a student exposes Coincall data publicly); hostile interaction. The mentor directs next steps.

4.5 Environment and Tooling

Each tutor receives Slack and LMS access, Coincall S3 credentials (under NDA, plus reference-implementation credentials), a desk in SAS Hall, and a laptop with the standard program environment pre-installed:

- **Python 3.11**, **PyTorch**, NumPy, SciPy, Pandas, PyArrow, and the program's own internal Python libraries.
- A **C++17 toolchain** (clang or g++), **CMake**, **Eigen**, a C++ **FFT** library (FFTW or pocketfft), and **pybind11** — used in Week 7 to build and import the fast-computation module from Python.
- Catch2 / GoogleTest and **pytest** for the test suites.

First-week setup: complete the data-use agreement; verify Slack/LMS access; build and import the Week-7 **pybind11** starter module end to end; run the reference Milestone 1 implementation; attend the calibration session.

4.6 Compensation, Hours, and Contacts

Tutors are compensated as standard graduate TAs under the NC State Mathematics pay scale; hours are tracked through the university system. The contact list (mentor, coordinator, departmental administrator, IT, emergency) is distributed at orientation — keep it accessible.